1. Introduction

A compiler is a system software that translates programs written in a high-level programming language into a lower-level representation that can be executed by a machine. One of the key phases in a compiler is Intermediate Code Generation, which transforms source code into an intermediate representation (IR) that is independent of both the source language and the target machine.

In this assignment, we design and implement a compiler front-end for a custom C-like language, focusing on the generation of Three Address Code (TAC) in quadruple format. The intermediate representation simplifies complex expressions and control flow constructs into a sequence of simple, structured instructions, making further processing and analysis easier.

The implemented compiler supports a variety of language features, including:

- Arithmetic and unary expressions
- Assignment statements
- Conditional constructs (`if`, `if-else`)
- Iterative constructs (`while` loops with control statements like `break`)
- Function definitions and function calls

Each construct in the source program is systematically translated into TAC using temporary variables and labels, ensuring that the semantics of the original program are preserved.

The primary objectives of this project are:

- To translate high-level language constructs into Three Address Code
- To handle control flow using conditional and unconditional jumps
- To represent programs in a structured quadruple format
- To demonstrate the working of a compiler pipeline up to the intermediate code generation phase

This project provides practical insight into compiler design and highlights how high-level abstractions are converted into lower-level representations through systematic translation techniques.

---

2. Language Specification

The compiler is designed for a custom C-like programming language with simplified syntax and a restricted set of features. The language is structured to support essential programming constructs while remaining easy to parse and translate into intermediate code.

2.1 Program Structure

A valid program in the language follows a defined structure:

- The program begins with the keyword `Accio`
- The program ends with the keyword `Lumos`
- Each statement is terminated using the symbol $

Example:

```
Accio

variable a : int$

a = 5$

Lumos
```

2.2 Variable Declaration

Variables must be declared before use using the following syntax:

```
variable <identifier> : <type>
```

Supported Data Types:

- `int`, `long`, `short`
- `float`, `double`
- `char`, `void`

2.3 Expressions and Operators

The language supports different types of operators :

- Arithmetic Operators:

  +, -, *, /, %

- Relational Operators:

  <, >, <=, >=, ==, !=

- Logical Operators:

  &&, ||

- Unary Operators:

  -, !, ~

Expressions can be combined using these operators and are evaluated based on standard precedence rules.

2.4 Assignment Statements

Assignment is performed using the = operator:

a = b + c$

2.5 Conditional Statements

The language supports decision-making using `if` and `if-else` constructs.

Syntax:

if (<condition>)

begin

...

end

if (<condition>)

begin

...

end

else

begin

...

end

2.6 Iterative Constructs

The language supports while loops for iteration:

while (<condition>)

begin

...

end

Additionally, the following loop control statements are supported:

- `break`
- `continue`

2.7 Functions

Functions can be declared using the `func` keyword:

func <return_type> <function_name>(parameters)

begin

...

end

Return Statement:

return expression$

2.8 Function Calls

Functions are invoked using the `call` keyword:

call <function_name>(arguments)$

2.9 Comments

- Single-line comments are supported
- Comments begin with the symbol #

2.10 Language Restrictions

To simplify the compiler design, the following features are not supported :

- No pointers or pointer arithmetic
- No switch-case statements
- No do-while or for loops
- No multi-line comments

2.11 Special Design Choices

The language introduces certain custom keywords and conventions:

- `Accio` → Marks the beginning of the program
- `Lumos` → Marks the end of the program
- `begin` and `end` → Used instead of { and }
- `$` → Used as a statement terminator

These design choices simplify parsing and improve readability for the compiler implementation.

---

3. System Architecture

The implemented system follows a modular compiler design, where the source program is processed through multiple stages to generate intermediate code in the form of Three Address Code (TAC).

The overall architecture of the compiler pipeline is as follows:

Source Program → Lexical Analyzer → Syntax Analyzer → Intermediate Code Generator

3.1 Overview of Components

The compiler consists of three major components:

1. Lexical Analyzer (Flex)

2. Syntax Analyzer (Yacc)

3. Intermediate Code Generator

Each component performs a specific role in transforming the input program into an intermediate representation.

3.2 Lexical Analyzer

The Lexical Analyzer, implemented using Flex, is responsible for scanning the input source code and converting it into a stream of tokens.

Key Responsibilities:

- Identifies keywords such as `if`, `while`, `func`, `call`
- Recognizes identifiers, constants, and operators
- Ignores whitespace and comments
- Detects invalid characters and reports lexical errors

Example:

variable a : int$

This is tokenized into:

- `variable` → keyword
- a → identifier
- `:` → delimiter
- `int` → type

3.3 Syntax Analyzer

The Syntax Analyzer, implemented using Yacc, processes the token stream and validates it against the grammar of the language.

Key Responsibilities:

- Ensures the program follows correct syntax
- Parses expressions, statements, and control structures
- Handles constructs such as:
    - assignments

    - conditional statements (`if`, `if-else`)
    - loops (`while`)
    - function definitions and calls
- Reports syntax errors when grammar rules are violated

3.4 Intermediate Code Generator

The **Intermediate Code Generator** is integrated with the parser using **Syntax Directed Translation (SDT)**.

Key Responsibilities:

- Generates **Three Address Code (TAC)**
- Uses **quadruple format** for representation
- Introduces:
    - temporary variables (`t1, t2, ...`)
    - labels (`L1, L2`) for control flow

Example Transformation:

High-level statement:

a = b + c * 2$

Generated TAC (Quadruple Format):

Idx	Op	Arg1	Arg2	Result
0	*	c	2	t1
1	+	b	t1	t2
2	=	t2		a

3.5 Data Flow Between Components

The processing of a program occurs as follows:

1. The source program is read line by line
2. The lexer converts it into tokens
3. The parser validates syntax and structure
4. During parsing, intermediate code is generated simultaneously
5. The final output is displayed in quadruple tabular format

3.6 Design Characteristics

- Modular design using Flex and Yacc
- Integration of parsing and code generation
- Clear separation of phases
- Structured output in TAC format

---

4. Intermediate Representation Design

The intermediate representation (IR) used in this compiler is Three Address Code (TAC) in the form of quadruples. This representation breaks down complex expressions and control structures into a sequence of simple instructions involving at most three operands.

4.1 Three Address Code (TAC)

Three Address Code is a linear representation of a program where each instruction contains:

- At most one operator
- At most two operands
- A result variable

This simplifies evaluation and makes the code suitable for further processing and analysis.

4.2 Quadruple Format

The TAC is represented using a quadruple structure with four fields:

Idx Op Arg1 Arg2 Result

Field Description:

- Idx → Instruction number
- Op → Operation to be performed
- Arg1 → First operand
- Arg2 → Second operand (if applicable)
- Result → Variable where the result is stored

4.3 Representation of Expressions

AArithmetic expressions are decomposed into smaller operations using temporary variables.

Example:

High-level expression:

a = b + c * 2$

Quadruple Representation:

Idx	Op	Arg1	Arg2	Result
0	*	c	2	t1
1	+	b	t1	t2
2	=	t2		a

4.4 Temporary Variables

Temporary variables (`t1, t2, t3, ...`) are used to store intermediate results during expression evaluation.

Characteristics:

- Generated automatically during parsing
- Ensure that each instruction has at most one operation
- Improve clarity and structure of IR

4.5 Control Flow Representation

Control flow constructs such as conditionals and loops are represented using labels and jump instructions.

Conditional Statements

Example:

if (a < b)

Quadruple Representation:

Idx	Op	Arg1	Arg2	Result
0	<	a	b	t1
1	ifFalse	t1		L1
2	=	b		a
3	label	L1		

Loop Constructs

Example:

while (i < 5)

Quadruple Representation:

Idx	Op	Arg1	Arg2	Result
0	label	L1		
1	<	i	5	t1
2	ifFalse	t1		L2

3	+	i	1	t2
4	=	t2		i
5	goto			L1
6	label	L2		

Control Instructions Used:

- `ifFalse` → Conditional jump
- `goto` → Unconditional jump
- `label` → Marks a position in code

4.6 Function Representation

Functions are represented using TAC instructions for computation and returning values.

Example:

return a + b$

Quadruple Representation:

Idx	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	return	t1		

4.7 Function Call Representation

Function calls are handled using parameter passing followed by a call instruction.

Example:

call sum(x, y)$

Quadruple Representation:

Idx	Op	Arg1	Arg2	Result
0	param	x		
1	param	y		
2	call	sum		

4.8 Key Features of the IR

- Simple and linear representation of programs
- Uses temporary variables for intermediate results
- Supports control flow using labels and jumps
- Machine-independent representation
- Easy to understand and debug

---

5. Implementation Details

The compiler is implemented using Flex (Lex) for lexical analysis and Yacc (Bison) for syntax analysis and intermediate code generation. The system follows a syntax-directed approach where parsing and code generation are integrated.

5.1 Lexical Analysis

The lexical analyzer is implemented using Flex and is responsible for scanning the input program and converting it into a stream of tokens.

Key Responsibilities:

- Identification of keywords:
  - `Accio`, `Lumos`, `if`, `else`, `while`, `func`, `call`, `return`, `break`, `continue`
- Recognition of:
  - identifiers
  - numeric constants
  - operators and delimiters
- Ignoring:

    - whitespace
    - single-line comments (#)

Error Detection:

The lexer detects invalid characters and reports lexical errors.

Example:

[Lexical Error] Line 3: Unknown character '@'

This ensures that only valid tokens are passed to the parser.

5.2 Syntax Analysis

The syntax analyzer is implemented using Yacc and validates the structure of the program based on the defined grammar.

Key Responsibilities:

- Parsing of:
    - variable declarations
    - assignment statements
    - expressions
    - conditional statements (`if`, `if-else`)
    - loops (`while`)
    - function definitions and calls
- Enforcement of correct program structure:
    - `Accio` as program start
    - `Lumos` as program end
    - `$` as statement terminator

Error Handling:

The parser reports syntax errors when grammar rules are violated.

Example:

[Syntax/Semantic Error] Line 4: syntax error near 'begin'

5.3 Intermediate Code Generation

Intermediate code generation is integrated within the parser using Syntax Directed Translation (SDT).As grammar rules are reduced, the corresponding Three Address Code (TAC) instructions are generated in quadruple format.

5.3.1 Expression Handling

Arithmetic and unary expressions are decomposed into smaller operations using temporary variables.

Example:

For the expression:

a = b + c * 2$

Generated TAC:

Idx	Op	Arg1	Arg2	Result
0	*	c	2	t1
1	+	b	t1	t2
2	=	t2		a

Each operation is translated into a separate instruction, ensuring that every step contains at most one operator.

5.3.2 Assignment Statements

Assignment statements are directly translated into TAC instructions.

Example:

a = t2$

Generated TAC:

Idx	Op	Arg1	Arg2	Result
0	=	t2		a

5.3.3 Conditional Statements

Conditional constructs are implemented using labels and conditional jumps.

Example:
if (a < b)

Generated TAC:

Idx	Op	Arg1	Arg2	Result
0	<	a	b	t1
1	ifFalse	t1		L1
2	=	b		a
3	label	L1		

5.3.4 Loop Constructs

Loops are implemented using a combination of labels and jump instructions.

Example:
while (i < 5)

Generated TAC:

Idx	Op	Arg1	Arg2	Result
0	label	L1		

1	<	i	5	t1
2	ifFalse	t1		L2
3	+	i	1	t2
4	=	t2		i
5	goto			L1
6	label	L2		

5.3.5 Control Statements (break, continue)

Control statements such as `break` and `continue` are handled using jump instructions.

- **break** transfers control outside the loop
- **continue** skips the remaining statements and proceeds to the next iteration

Example (break):

```
if (i == 5)
  break$
```

Generated TAC:

Idx	Op	Arg1	Arg2	Result
0	==	i	5	t1
1	ifFalse	t1		L1
2	goto			L2
3	label	L1		

5.3.6 Function Handling

Functions are translated into TAC by generating instructions for computation and returning values.

Example:

```
return a + b$
```

Generated TAC:

Idx	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	return	t1		

5.3.7 Function Calls

Function calls are implemented using parameter passing followed by a call instruction.

Example:

call sum(x, y)$

Generated TAC:

Idx	Op	Arg1	Arg2	Result
0	param	x		
1	param	y		
2	call	sum		

5.4 Temporary Variables and Labels

- Temporary variables (`t1, t2, ...`)
  - Used for intermediate computations
  - Generated dynamically during parsing
- Labels (`L1, L2, ...`)
  - Used to represent control flow
  - Mark positions for conditional and unconditional jumps

5.5 Output Generation

The generated intermediate code is displayed in a tabular quadruple format, ensuring clarity and readability.

Format:

| Idx | Op | Arg1 | Arg2 | Result |

- Instructions are printed sequentially
- Temporary variables and labels are clearly represented

- Output matches the required format specified in the assignment

---

6. Code Generation Examples (10 Programs)

This section presents ten sample programs written in the designed language along with their corresponding intermediate code in quadruple format. Each example demonstrates how high-level constructs are translated into Three Address Code (TAC).

Program 1: Simple Assignment

Input Source Program:

```
Accio
variable a : int$
a = 5$
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	=	5		a

Program 2: Arithmetic Expression with Precedence

Input Source Program:

```
Accio
variable a : int$
variable b : int$
variable c : int$
a = b + c * 2$
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	*	c	2	t1
1	+	b	t1	t2
2	=	t2		a

Program 3: Unary Expression

Input Source Program:

```
Accio
variable x : int$
variable y : int$
x = -y + 10$
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	minus	y		t1
1	+	t1	10	t2
2	=	t2		x

Program 4: If Statement

Input Source Program:

```
Accio
variable a : int$
variable b : int$
if (a < b)
begin
a = b$
end
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	<	a	b	t1
1	ifFalse	t1		L1
2	=	b		a
3	label	L1		

Program 5: If-Else Statement

Input Source Program:

```
Accio
variable a : int$
variable b : int$
if (a > b)
begin
a = a - b$
end
else
begin
b = b - a$
end
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	>	a	b	t1
1	ifFalse	t1		L1
2	-	a	b	t2
3	=	t2		a
4	goto			L2
5	label	L1		
6	-	b	a	t3
7	=	t3		b

8	label	L2		

Program 6: While Loop

Input Source Program:

```
Accio
variable i : int$
i = 0$
while (i < 5)
begin
i = i + 1$
end
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	=	0		i
1	label	L1		
2	<	i	5	t1
3	ifFalse	t1		L2
4	+	i	1	t2
5	=	t2		i
6	goto			L1
7	label	L2		

Program 7: While Loop with Break

Input Source Program:

```
Accio
variable i : int$
i = 0$
while (i < 10)
begin
```

```
if (i == 5)
begin
break$
end
i = i + 1$
end
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	=	0		i
1	label	L1		
2	<	i	10	t1
3	ifFalse	t1		L2
4	==	i	5	t2
5	ifFalse	t2		L3
6	goto			L2
7	label	L3		
8	+	i	1	t3
9	=	t3		i
10	goto			L1
11	label	L2		

Program 8: Function Definition

Input Source Program:

```
Accio
func int sum(a : int, b : int)
begin
return a + b$
end$
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	return	t1		

Program 9: Function Call

Input Source Program:

```
Accio
variable x : int$
variable y : int$
call sum(x, y)$
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	param	x		
1	param	y		
2	call	sum		

Program 10: Combined Program

Input Source Program:

```
Accio
func int sum(a : int, b : int)
begin
return a + b$
end$
variable x : int$
variable y : int$
x = 5$
y = 10$
call sum(x, y)$
Lumos
```

Generated Intermediate Code:

Idx	Op	Arg1	Arg2	Result
0	+	a	b	t1
1	return	t1		
2	=	5		x
3	=	10		y
4	param	x		
5	param	y		
6	call	sum		

---

7. Error Handling

Error handling is an essential component of a compiler to ensure that invalid inputs are detected and reported in a meaningful manner without causing the system to crash. The implemented compiler provides basic error detection at both the lexical and syntax/semantic levels.

7.1 Lexical Error Handling

The lexical analyzer (Flex) is responsible for identifying tokens. If an invalid or unknown character is encountered, it reports a lexical error.

Example

Input Source Program:

```
Accio

variable x : int$

x = 10 @ 5$

Lumos
```

Output:

[Lexical Error] Line 3: Unknown character '@'

As observed in execution , the lexer correctly detects the invalid symbol @, which is not part of the language specification.

7.2 Syntax and Semantic Error Handling

The parser (Yacc) detects errors related to incorrect grammar, malformed expressions, and invalid statement structures. These are reported as syntax/semantic errors.

Example 1: Invalid Expression

Input Source Program:

```
Accio
variable x : int$
x = 10 @ 5$
Lumos
```

Output:

[Syntax/Semantic Error] Line 3: syntax error near '5'

Parsing failed due to errors.

Here, after the lexical error, the parser also detects that the expression structure is invalid.

Example 2: Missing Parenthesis / Invalid Structure

Input Source Program:

```
Accio
variable a : int$
if (a < 10
begin
a = $
end
Lumos
```

Output:

[Syntax/Semantic Error] Line 4: syntax error near 'begin'

Parsing failed due to errors.

As shown in execution , the parser identifies:

- missing closing parenthesis
- incomplete assignment expression

7.3 Error Reporting Strategy

The compiler follows a **fail-safe error handling strategy**:

- Errors are reported with:
    - Type of error (Lexical / Syntax)
    - Line number
    - Approximate location of error
- After encountering errors:
    - The compiler **does not crash**
    - Execution terminates gracefully with the message:
      Parsing failed due to errors.

7.4 Observations

- Lexical and syntax errors are clearly distinguished.
- Error messages are informative and help locate the issue.
- The compiler stops code generation when critical errors are detected, ensuring correctness of output.

7.5 Limitations of Error Handling

- No advanced error recovery (parser does not continue after major errors)
- Limited semantic checks (e.g., type mismatch not fully handled)
- Only first-level error reporting (no cascading error analysis)

---

8. Limitations

While the compiler successfully generates intermediate code for a wide range of constructs, there are several limitations in the current implementation.

8.1 Lack of Code Optimization

The generated intermediate code is not optimized.

- Redundant computations are not eliminated
- No constant folding or dead code elimination is performed
- Expressions are translated directly into TAC without simplification

For example, repeated sub-expressions are recomputed instead of being reused.

8.2 Limited Type Checking

Although variable declarations are supported, comprehensive type checking is not implemented.

- Operations between incompatible types are not fully validated
- No strict enforcement of type consistency across expressions
- Function argument type matching is not rigorously checked

8.3 Basic Error Handling

The compiler provides basic error detection, but lacks advanced recovery mechanisms.

- Parsing stops after encountering errors
- No attempt is made to continue parsing after an error
- Limited semantic error reporting

As observed in execution , the compiler terminates after reporting syntax errors.

8.4 Restricted Language Features

The designed language intentionally supports a limited subset of C-like constructs :

- No support for pointers
- No switch-case statements
- No do-while or for loops
- No multi-line comments

These restrictions simplify implementation but reduce expressiveness.

8.5 No Symbol Table Enhancements

The current implementation does not include advanced symbol table features such as:

- Scope management (nested scopes, block-level scope)
- Variable lifetime tracking
- Function overloading support

8.6 No Intermediate Code Optimization or Backend

The compiler is limited to front-end processing:

- Only generates intermediate code (TAC)
- No conversion to machine code or assembly
- No register allocation or instruction selection

---

9. Conclusion

In this assignment, we successfully designed and implemented a compiler front-end for a custom C-like language, focusing on Intermediate Code Generation using Three Address Code (TAC) in quadruple format.

The compiler is capable of handling a variety of programming constructs, including:

- Arithmetic and unary expressions
- Assignment statements
- Conditional statements (`if`, `if-else`)
- Iterative constructs (`while` loops with `break`)
- Function definitions and function calls

High-level constructs are systematically translated into a structured intermediate representation using temporary variables and control flow labels. The generated TAC accurately reflects the semantics of the input programs, as demonstrated through multiple test cases .

The implementation integrates lexical analysis, syntax analysis, and intermediate code generation into a cohesive pipeline, demonstrating the working of a compiler up to the IR phase.

Although the system has certain limitations, such as lack of optimization and limited type checking, it effectively achieves the objective of translating source programs into a clear and structured intermediate form.

Overall, the project provides practical insight into compiler design principles and demonstrates a functional implementation of intermediate code generation for a simplified programming language.

---

10. End-to-End Pipeline and Team Contribution

The implemented compiler follows a complete front-end pipeline that transforms a source program into intermediate code in Three Address Code (TAC) format.

Pipeline Overview:

Source Program → Lexical Analysis → Syntax Analysis → Intermediate Code Generation → TAC Output

- The **lexical analysis phase** tokenizes the input program and detects invalid characters.
- The **syntax analysis phase** validates the structure of the program using grammar rules.
- During parsing, **intermediate code generation** is performed using syntax-directed translation, producing TAC with temporary variables and labels.
- The final output is displayed in a **quadruple tabular format**, as required.

Team Contribution

The project was developed collaboratively, with contributions spanning all stages of the compiler pipeline. The team collectively worked on:

- Designing the language grammar and structure
- Implementing lexical and syntax analysis using Flex and Yacc
- Developing intermediate code generation using Three Address Code
- Testing the compiler with multiple input programs
- Preparing documentation and organizing outputs